Unit 6
Using MySQL/MSSQL Server



Introduction

What is SQL?

SQL (pronounced "ess-que-el") stands for Structured Query Language. SQL is used to communicate with a database. According to ANSI (American National Standards Institute), it is the standard language for relational database management systems. SQL statements are used to perform tasks such as update data on a database, or retrieve data from a database. Some common relational database management systems that use SQL are: Oracle, Sybase, Microsoft SQL Server, Access, Ingres, etc. Although most database systems use SQL, most of them also have their own additional proprietary extensions that are usually only used on their system. However, the standard SQL commands such as "Select", "Insert", "Update", "Delete", "Create", and "Drop" can be used to accomplish almost everything that one needs to do with a database. This tutorial will provide you with the instruction on the basics of each of these commands as well as allow you to put them to practice using the SQL Interpreter.

What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

SQL is a Standard - BUT....

Although SQL is an ANSI/ISO standard, there are different versions of the SQL language.

However, to be compliant with the ANSI standard, they all support at least the major commands (such as SELECT, UPDATE, DELETE, INSERT, WHERE) in a similar manner.

Note: Most of the SQL database programs also have their own proprietary extensions in addition to the SQL standard!

Using SQL in Your Web Site

To build a web site that shows data from a database, you will need:

- An RDBMS database program (i.e. MS Access, SQL Server, MySQL)
- To use a server-side scripting language, like PHP or ASP
- To use SQL to get the data you want
- To use HTML / CSS to style the page

RDBMS

RDBMS stands for Relational Database Management System.

RDBMS is the basis for SQL, and for all modern database systems such as MS SQL Server, IBM DB2, Oracle, MySQL, and Microsoft Access.



Unit Learning Outcomes

At the end of the unit, students will be able to:

1. become familiar with the different SQL Statements in data manipulation
2. insert and modify data in MySQL/MSSQL Server database.
3. write sub queries to retrieve data.
4. retrieve data using different SQL clauses

Topic 1: Inserting, Deleting and Updating Data

Time Allotment: 2 Hour



Learning Objectives

At the end of the topic, students will be able to:

1. Know and familiarize the syntax of insert, delete and update statement in SQL
2. differentiate insert, delete, update SQL Statements
3. formulate SQL statements in inserting, deleting and updating data in database



Activating Prior Knowledge

List down the things that you know, you want to learn and you have learned about this topic on inserting, deleting and updating data in database table.

What you know	What you want to learn	What you have learned



Presentation of Contents

The SQL INSERT INTO Statement

The **INSERT INTO** statement is used to insert new records in a table.

INSERT INTO Syntax

It is possible to write the INSERT INTO statement in two ways.

The first way specifies both the column names and the values to be inserted:

INSERT INTO *table_name* (*column1*, *column2*, *column3*, ...)
VALUES (*value1*, *value2*, *value3*, ...);

If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table. The INSERT INTO syntax would be as follows:

INSERT INTO *table_name*
VALUES (*value1*, *value2*, *value3*, ...);

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT

	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS

INSERT INTO Example

The following SQL statement inserts a new record in the "tblStudents" table:

Example

```
INSERT INTO tblStudents (StudentNo, StudentName, Address, Gender, Birthday, Course)
VALUES ('19-09278', 'Eidan Deeandrei', 'Sanchez Mira', 'Male', '04/27/2001', 'BMMA');
```

The demo table "tblStudents" will now look like this:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA

Note: The record in red text was new inserted after executing the sample insert query above.

Did you notice that we did not insert any number into the StudentID field?

The StudentID column is an auto-increment field and will be generated automatically when a new record is inserted into the table.

Insert Data Only in Specified Columns

It is also possible to only insert data in specific columns.

The following SQL statement will insert a new record, but only insert data in the "StudentName", "Address", and "Course" columns (StudentID will be updated automatically):

Example

INSERT INTO tblStudents (StudentName, Address, Course) VALUES ('18-12345', 'Dane Arviel' 'BSCrim');

The demo table "tblStudents" will now look like this:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim

Note: The record in red text was new inserted after executing the sample insert query above.

Insert Data without specifying column/s

It is also possible to only insert data without specifying column/s.

The following SQL statement will insert a new record with complete entry in each of the columns. (StudentID will be updated automatically):

Example

INSERT INTO tblStudents VALUES ('17-23456', Juana Dela Cruz' 'Kabugnayan Village', 'Female', '02/14/2002', 'BSE');

The demo table "tblStudents" will now look like this:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS

3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

Note: The record in red text was new inserted after executing the sample insert query above.

The SQL DELETE Statement

The DELETE statement is used to delete existing records in a table.

DELETE Syntax

DELETE FROM *table_name* WHERE *condition*;

Note: Be careful when deleting records in a table! Notice the WHERE clause in the DELETE statement. The WHERE clause specifies which record(s) should be deleted. If you omit the WHERE clause, all records in the table will be deleted!

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

SQL DELETE Example

The following SQL statement deletes the student " Juana Dela Cruz " from the "tblStudents" table:

Example

DELETE FROM tblStudents WHERE StudentName= 'Juana Dela Cruz';

The demo table "tblStudents" will now look like this:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim

Note: Record #5 was deleted because that was the record of Juana Dela Cruz which was deleted in the previous Delete statement.

Delete All Records

It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact:

DELETE FROM *table_name*;

The following SQL statement deletes all rows in the "tblStudents" table, without deleting the table:

Example

DELETE FROM Customers;

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course

Note: Empty set or table because all records was deleted after executing the previous Delete statement.

The SQL UPDATE Statement

The UPDATE statement is used to modify the existing records in a table.

UPDATE Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2,                    ...
WHERE condition;
```

Note: Be careful when updating records in a table! Notice the WHERE clause in the UPDATE statement. The WHERE clause specifies which record(s) that should be updated. If you omit the WHERE clause, all records in the table will be updated!

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

UPDATE Table

The following SQL statement updates the first student (StudentID = 1) with a new **address** *and* a new **course**.

Example

```
UPDATE tblStudents
SET Address = 'Nagbaranganan City', Course= 'BSA'
WHERE StudentID = 1;
```

The demo table "tblStudents" will now look like this:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Nagbaranganan City	Male	08/27/2000	BSA
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

Note: Record number 1 was updated where its Address and Course was changed into new values which are in red text.

UPDATE Multiple Records

It is the WHERE clause that determines how many records will be updated.

The following SQL statement will update the Course to "BSIT" for all records where Gender is "Male":

Example

```
UPDATE tblStudents
SET Course='BSIT'
WHERE Gender='Male';
```

The demo table "tblStudents" will now look like this:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course

1	18-14327	Gorgonio Magalpok	Nagbaranganan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIT
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BSIT
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

Note: There were 3 records affected where Gender = Male, and were updated setting Course to BSIT. Notice the data in red text.

Update Warning!

Be careful when updating records. If you omit the WHERE clause, ALL records will be updated!

Example

```
UPDATE tblStudents
SET Course='BSIT';
```

The demo table "tblStudents" will now look like this:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Nagbaranganan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIT
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BSIT

4	18-12345	Dane Arviel	Null	Null	Null	BSIT
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSIT

Note: All records were updated setting Course = BSIT because in the previous Update query, we omit where clause meaning, all records in the table will be updated.



Application

Given: tblActivity; assumed this as one table in dbClassRecord database

RecID	StudNo	StudName	Score	Grade

Instructions: Fill in the needed words in each blank to complete the query to produce an accurate result.

(Insert new record in the table)

1. Insert into _____ (StudNo, _____, Score, Grade) _____ ('08-74453','Cianong Dimakatugal',10,100).

(Update record which was recently added in the table setting Score = 5 and Grade = 75)

2. Update tblActivity _____ Score = ____, _____ = 75 where _____ = 1

(Delete record number 1)

3. _____ from tblActivity _____ _____= 1



Feedback

Formulate a query based on the given instructions below. Consider the design of the table **tblFeedback1**.

StudID	Lname	Fname	Mname	Score	Grade	Remarks

Note: StudID is an auto_increment field, Lname, Fname, Mname, Remarks are nvarchar fields, Score and Grade are numeric fields. (2pts each)

1. Insert a record considering your complete name with a score of 15, grade of 75 and remarks = Passed.

Answer:_____
_____

2. Insert another record considering the complete name of your subject teacher with a score of 30, grade of 100 and remarks = Passed.

Answer:

_____
_____

3. Update record #1 changing the score = 0, grade = 70 and remarks = Failed.
4. Delete record/s where grade >= 100 and remarks = Failed.
5. Fill up the table with correct records based on the 4 SQL statements executed above.

StudID	Lname	Fname	Mname	Score	Grade	Remarks



Reflection

Inserting is usually a straightforward task. It begins with the simple problem of inserting a single row. Many times, however, it is more efficient to use a set-based approach to create new rows. To that end, you'll also find techniques for inserting many rows at a time.

Likewise, updating and deleting start out as simple tasks. You can update one record, and you can delete one record. But you can also update whole sets of records at once, and in very powerful ways. And there are many handy ways to delete records. For example, you can delete rows in one table depending on whether or not they exist in another table.

Topic 2: OVERVIEW OF SELECT STATEMENT

Time Allotment: 1 Hour



Learning Objectives

At the end of the topic, students will be able to:

1. know and familiarize the syntax of select SQL statement
2. know how to formulate a condition based on needed data
3. use select by sorting the records in least to greatest or vice versa
4. formulate SQL statements in selecting record/s of the table based on different conditions.

Activating Prior Knowledge

List down different keywords used in SELECT SQL Statement

1. _____ 6. _____
2. _____ 7. _____
3. _____ 8. _____
4. _____ 9. _____
5. _____ 10. _____

Presentation of Contents

The SQL SELECT Statement

The SELECT statement is used to select data from a database.

The data returned is stored in a result table, called the result-set.

SELECT Syntax

SELECT *column1*, *column2,...*
FROM *table_name*;

Here, column1, column2, ... are the field names of the table you want to select data from. If you want to select all the fields available in the table, use the following syntax:

SELECT * FROM *table_name*;

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim

5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

SELECT Column Example

The following SQL statement selects the "StudentName" and "Gender" columns from the "tblStudents" table:

Example

```
SELECT StudentName, Gender FROM tblStudents;
```

SELECT * Example

The following SQL statement selects all the columns from the "tblStudents" table:

Example

```
SELECT * FROM tblStudents;
```

The SQL SELECT DISTINCT Statement

The SELECT DISTINCT statement is used to return only distinct (different) values.

Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.

SELECT DISTINCT Syntax

```
SELECT DISTINCT column1, column2,                    ...
FROM table_name;
```

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS

3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

SELECT Example Without DISTINCT

The following SQL statement selects ALL (including the duplicates) values from the "Course" column in the "tblStudents" table:

Example

```
SELECT Course FROM tblStudents;
```

Output:

Course
BSIT
BSIS
BMMA
BSCrim
BSE

Now, let us use the DISTINCT keyword with the above SELECT statement and see the result.

SELECT DISTINCT Examples

The following SQL statement selects only the DISTINCT values from the "Course" column in the "tblStudents" table:

Example

```
SELECT DISTINCT Course FROM tblStudents;
```

The SQL SELECT TOP Clause

The SELECT TOP clause is used to specify the number of records to return.

The SELECT TOP clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

Note: Not all database systems support the SELECT TOP clause. MySQL supports the LIMIT clause to select a limited number of records, while Oracle uses ROWNUM.

SQL Server / MS Access Syntax:

```
SELECT TOP number|percent column_name(s)
FROM table_name
WHERE condition;
```

MySQL Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE condition
LIMIT number;
```

Oracle Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE ROWNUM <= number;
```

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA

4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

SQL TOP, LIMIT and ROWNUM Examples

The following SQL statement selects the first three records from the "tblStudents" table (for SQL Server/MS Access):

Example

```
SELECT TOP 3 * FROM tblStudents;
```

The following SQL statement shows the equivalent example using the LIMIT clause (for MySQL):

Example

```
SELECT * FROM tblStudents
LIMIT 3;
```

The following SQL statement shows the equivalent example using ROWNUM (for Oracle):

Example

```
SELECT * FROM tblStudents
WHERE ROWNUM <= 3;
```

SQL TOP PERCENT Example

The following SQL statement selects the first 50% of the records from the " tblStudents " table (for SQL Server/MS Access):

Example

```
SELECT TOP 50 PERCENT * FROM tblStudents;
```

ADD a WHERE CLAUSE

The following SQL statement selects the first three records from the " tblStudents " table, where the gender is "Female" (for SQL Server/MS Access):

Example

```
SELECT TOP 3 * FROM tblStudents
WHERE Gender='Female';
```

The following SQL statement shows the equivalent example using the LIMIT clause (for MySQL):

Example

```
SELECT * FROM tblStudents
WHERE Course='BSIT'
LIMIT 3;
```

The following SQL statement shows the equivalent example using ROWNUM (for Oracle):

Example

```
SELECT * FROM tblStudents
WHERE Course='BSIT' AND ROWNUM <= 3;
```

The SQL SELECT INTO Statement

The SELECT INTO statement copies data from one table into a new table.

SELECT INTO Syntax

Copy all columns into a new table:

```
SELECT *
INTO newtable [IN externaldb]
FROM oldtable
WHERE condition;
```

Copy only some columns into a new table:

```
SELECT column1, column2, column3,                    ...
INTO newtable [IN externaldb]
FROM oldtable
WHERE condition;
```

The new table will be created with the column-names and types as defined in the old table. You can create new column names using the AS clause.

SQL SELECT INTO Examples

The following SQL statement creates a backup copy of tblStudents:

```
SELECT * INTO StudentsBackup2020
FROM tblStudents;
```

The following SQL statement uses the IN clause to copy the table into a new table in another database:

```
SELECT * INTO StudentsBackup2020 IN 'Backup.mdb'
FROM tblStudents;
```

The following SQL statement copies only a few columns into a new table:

```
SELECT StudentName,                Course INTO StudentsBackup2020
FROM tblStudents;
```

The following SQL statement copies only the BSIT students into a new table:

```
SELECT * INTO BSITStudents
FROM tblStudents
WHERE Course = 'BSIT';
```

The following SQL statement copies data from more than one table into a new table:

```
SELECT tblStudents.StudentName, Course.CourseID
INTO StudentsBackup2020
FROM tblStudents
LEFT JOIN Course ON tblStudents.StudentID = Course.CourseID;
```

Tip: SELECT INTO can also be used to create a new, empty table using the schema of another. Just add a WHERE clause that causes the query to return no data:

```
SELECT * INTO newtable
FROM oldtable
WHERE 1 = 0;
```



Application

Given: tblActivity; assumed this as one table in dbClassRecord database

RecID	StudNo	StudName	Score	Grade
1	14-14327	Magalpok, Gorgonio	10	100
2	17-02020	Dela Cruz, Juan	5	75
3	19-89455	Dimakatugkal, Mildred	9	90

Instructions: Fill in the needed words in each blank to complete the query and produce an accurate result.

1. Select (_____, _____) from tblActivity.
   Result:

StudNo	StudName
14-14327	Magalpok, Gorgonio
17-02020	Dela Cruz, Juan
19-89455	Dimakatugkal, Mildred

2. Select * _____ tblActivity _____ Grade > _____.
   Result:

RecID	StudNo	StudName	Score	Grade
1	14-14327	Magalpok, Gorgonio	10	100
3	19-89455	Dimakatugkal, Mildred	9	90

3. Select _____, Grade from _____ where Grade >=100 order by _____ asc.

   Result:

StudName	Score
Dela Cruz, Juan	5
Dimakatugkal, Mildred	9
Magalpok, Gorgonio	10

Feedback

tblFeedback2

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim

5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

Considering the table above, formulate SQL Statements on the following instructions below.

1. Select all records in the table considering all fields sorting StudentName is ascending order.

   _____
   _____

2. Select record/s in the table considering StudentNo, StudentName and Course where course = 'BSIT'.

   _____
   _____

3. Select record/s in the table considering all fields where gender = 'Male' sorting StudentName in ascending order.

   _____
   _____

Topic 3: Using WHERE clause to SELECT a particular row

Time Allotment: 1 Hour

Learning Objectives

At the end of the topic, students will be able to:

1. know and familiarize the syntax of select SQL statement by incorporating WHERE clause
2. know how to formulate a condition in using WHERE clause
3. formulate SQL statements in selecting record/s of the table based on different conditions using WHERE clause.

Activating Prior Knowledge

1. Why do we need to use WHERE clause in selecting record/s of a table?

_____

2. Using WHERE clause in select SQL statement, what should be formulated to come up with a satisfying result?

_____

3. Formulate Select SQL statement with WHERE clause on this instruction.

*** Select record/s considering all the fields of the tblSample where grade >=75 sorting grade in descending order.*



Presentation of Contents

The SQL **WHERE** clause is used to specify a condition while fetching the data from a single table or by joining with multiple tables. If the given condition is satisfied, then only it returns a specific value from the table. You should use the WHERE clause to filter the records and fetching only the necessary records.

The WHERE clause is not only used in the SELECT statement, but it is also used in the UPDATE, DELETE statement, etc., which we would examine in the subsequent chapters.

Syntax

The basic syntax of the SELECT statement with the WHERE clause is as shown below.

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition]
```

You can specify a condition using the comparison or logical operators like >, <, =, **LIKE, NOT**, etc. The following examples would make this concept clear.

Example

Consider the CUSTOMERS table having the following records −

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
|  2 | Khilan   |  25 | Delhi     |  1500.00 |
|  3 | kaushik  |  23 | Kota      |  2000.00 |
|  4 | Chaitali |  25 | Mumbai    |  6500.00 |
|  5 | Hardik   |  27 | Bhopal    |  8500.00 |
|  6 | Komal    |  22 | MP        |  4500.00 |
|  7 | Muffy    |  24 | Indore    | 10000.00 |
+----+----------+-----+-----------+----------+
```

The following code is an example which would fetch the ID, Name and Salary fields from the CUSTOMERS table, where the salary is greater than 2000 −

```
SQL> SELECT ID, NAME, SALARY
FROM CUSTOMERS
```

```
WHERE SALARY > 2000;
```

This would produce the following result −

```
+----+----------+----------+
| ID | NAME     | SALARY   |
+----+----------+----------+
|  4 | Chaitali |  6500.00 |
|  5 | Hardik   |  8500.00 |
|  6 | Komal    |  4500.00 |
|  7 | Muffy    | 10000.00 |
+----+----------+----------+
```

The following query is an example, which would fetch the ID, Name and Salary fields from the CUSTOMERS table for a customer with the name **Hardik**.

Here, it is important to note that all the strings should be given inside single quotes ('). Whereas, numeric values should be given without any quote as in the above example.

```
SQL> SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE NAME = 'Hardik';
```

This would produce the following result −

```
+----+----------+----------+
| ID | NAME     | SALARY |
+----+----------+----------+
|  5 | Hardik   |  8500.00    |
+----+----------+----------+
```

Operators in The WHERE Clause

The following operators can be used in the WHERE clause:

Operator	Description
=	Equal
>	Greater than

<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column



Application

1. Select all records where the `Town` column has the value "Sanchez Mira".

```
SELECT * FROM tblCustomers _____________
_________ = _____________;
```

2. Use the `NOT` keyword to select all records where `Town` is NOT "Sanchez Mira".

```
SELECT * FROM tblCustomers _______________ =
"_____________";
```

3. Select all records where the `StudentID` column has the value 32.

```
SELECT * FROM tblCustomers _______ StudentID
_____ _____;
```

4. Select all records where the `Town` column has the value 'Sanchez Mira' *and* the `ZipCode` column has the value 3518.

```
___________ * FROM tblCustomers ________ Town =
"Sanchez Mira" _____ ________________ = _______;
```

5. Select all records where the `Town` column has the value 'Sanchez Mira' or 'Lasam'.

```
___________ * FROM tblCustomers ________ Town =
"Sanchez Mira" _____ ________________ = "_______";
```



Feedback

1. What is the meaning of the "WHERE" clause in SQL Statement?
a) Filtering out unwanted rows from result set
b) Filtering out unwanted columns from result set
c) Filtering out unwanted rows and columns from result set
d) None of the mentioned

2. What will be the output of the following SQL command?

```
SELECT *
FROM employee
WHERE title='HEAD TELLER';
```

a) All columns and rows belong to table employee
b) All columns but only those rows which contain 'HEAD TELLER' as a "title"
c) All columns don't belong to table employee
d) None of the mentioned

3. What will be the output of the following SQL command?

```
SELECT *
FROM employee
WHERE (title='HEAD TELLER') AND (start_date=2013-01-24);
```

a) All columns and rows belong to table employee
b) All columns but only those rows which contain 'HEAD TELLER' as a "title" and 2013-01-24 as a "start_date"
c) All rows belong to table employee
d) None of the mentioned

4. What will be the output of the following SQL command?

```
SELECT *
FROM employee
WHERE (title='HEAD TELLER') OR (start_date=2013-01-24);
```

a) All columns and rows belong to table employee
b) All columns but only those rows which contain 'HEAD TELLER' as a "title" or 2013-01-24 as a "start_date"
c) All rows belong to table employee
d) None of the mentioned

5. What will be the output of the following SQL command?

```
SELECT *
FROM employee
WHERE (title='HEAD TELLER') AND (start_date>;2013-01-24);
```

a) All columns and rows belong to table employee
b) All columns but only those rows which contain 'HEAD TELLER' as a "title" and start_date are greater than 2013-01-24
c) All rows belong to table employee
d) None of the mentioned

6. What will be the output of the following MySQL command?

```
SELECT *
FROM employee
WHERE (title='HEAD TELLER') OR (start_date>2013-01-24);
```

a) All columns and rows belong to table employee
b) All columns but only those rows which contain 'HEAD TELLER' as a "title" OR start_date are greater than 2013-01-24
c) All rows belong to table employee
d) None of the mentioned

7. What will be the output of the following SQL command?

```
SELECT emp_id, fname, lname
FROM employee
WHERE title='HEAD TELLER';
```

a) All columns
b) Only those columns which are mention with "SELECT" clause
c) Columns mention with "SELECT" clause and only those rows which contain 'HEAD TELLER' as a "title"
d) None of the mentioned

8. What will be the output of the following MySQL command?

```
SELECT emp_id, fname, lname
FROM employee
WHERE title='HEAD TELLER' AND start_date=2008-11-24;
```

a) All columns
b) Only those columns which are mention with “SELECT” clause
c) Columns mention with “SELECT” clause and only those rows which contain ‘HEAD TELLER’ as a “title” and start_date as 2008-11-24
d) None of the mentioned

9. What will be the output of the following MySQL command?

```
SELECT emp_id, fname, lname
FROM employee
WHERE title='HEAD TELLER' AND start_date>2008-11-23;
```

a) All columns
b) Only those columns which are mention with “SELECT” clause
c) Columns mention with “SELECT” clause and only those rows which contain ‘HEAD TELLER’ as a “title” and start_date>2008-11-23
d) None of the mentioned

Topic 4: Removing Duplicates with DISTINCT

Time Allotment: 1 Hour

Learning Objectives

At the end of the topic, students will be able to:

1. know and familiarize the syntax of select SQL statement by incorporating DISTINCT clause.
2. know how to remove duplicates by using DISTINCT clause.
3. formulate SQL statements in selecting unique record/s of the table using DISTINCT clause.

Activating Prior Knowledge

Sample

Presentation of Contents

The SQL SELECT DISTINCT Statement

The SELECT DISTINCT statement is used to return only distinct (different) values.

Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.

SELECT DISTINCT Syntax

```
SELECT DISTINCT column1, column2,                    ...
FROM table_name;
```

Demo Database

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

SELECT Example Without DISTINCT

The following SQL statement selects ALL (including the duplicates) values from the "Address" column in the "tblStudents" table:

Example

```
SELECT Address FROM tblStudents;
```

Now, let us use the DISTINCT keyword with the above SELECT statement and see the result.

SELECT DISTINCT Examples

The following SQL statement selects only the DISTINCT values from the "Address" column in the "tblStudents" table:

Example

```
SELECT DISTINCT Address FROM tblStudents;
```

The following SQL statement lists the number of different (distinct) student addresses:

Address
Di Masarsarakan City
Quezon City
Sanchez Mira
Kabugnayan Village

Example

```
SELECT COUNT(DISTINCT Address) FROM tblStudents;
```

Note: The example above will not work in MS Access! Because COUNT(DISTINCT *column_name*) is not supported in Microsoft Access databases.

Another Example

Consider the CUSTOMERS table having the following records −

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
|  2 | Khilan   |  25 | Delhi     |  1500.00 |
|  3 | kaushik  |  23 | Kota      |  2000.00 |
|  4 | Chaitali |  25 | Mumbai    |  6500.00 |
|  5 | Hardik   |  27 | Bhopal    |  8500.00 |
|  6 | Komal    |  22 | MP        |  4500.00 |
|  7 | Muffy    |  24 | Indore    | 10000.00 |
+----+----------+-----+-----------+----------+
```

First, let us see how the following SELECT query returns the duplicate salary records.

```
SQL> SELECT SALARY FROM CUSTOMERS
   ORDER BY SALARY;
```

This would produce the following result, where the salary (2000) is coming twice which is a duplicate record from the original table.

```
+----------+
| SALARY   |
+----------+
```

```
|  1500.00 |
|  2000.00 |
|  2000.00 |
|  4500.00 |
|  6500.00 |
|  8500.00 |
| 10000.00 |
+----------+
```

Now, let us use the DISTINCT keyword with the above SELECT query and then see the result.

```
SQL> SELECT DISTINCT SALARY FROM CUSTOMERS
  ORDER BY SALARY;
```

This would produce the following result where we do not have any duplicate entry.

```
+----------+
| SALARY   |
+----------+
|  1500.00 |
|  2000.00 |
|  4500.00 |
|  6500.00 |
|  8500.00 |
| 10000.00 |
+----------+
```



Application

Fill in the blanks.

1. Select all the different values from the `Town` column in the `tblCustomers` table.

   ```
   _____________ ________ Country FROM
   tblCustomers;
   ```



Feedback

tblCustomer

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
|  2 | Khilan   |  25 | Delhi     |  1500.00 |
|  3 | kaushik  |  23 | Kota      |  2000.00 |
|  4 | Chaitali |  25 | Mumbai    |  6500.00 |
|  5 | Hardik   |  27 | Bhopal    |  8500.00 |
|  6 | Komal    |  22 | MP        |  4500.00 |
|  7 | Muffy    |  24 | Indore    | 10000.00 |
+----+----------+-----+-----------+----------+
```

1. Formulate an SQL Statement to select distinct SALARY sorting SALARY in desc order.

   _____
   _____

2. Formulate an SQL Statement to select distinct NAME sorting NAME in ascending order.

   _____
   _____

3. Formulate an SQL Statement to select distinct AGE and ADDRESS sorting AGE in ascending order.

   _____
   _____

Topic 5: Using GROUP BY clause

Time Allotment: 1 Hour



Learning Objectives

At the end of the topic, students will be able to:

1. know and familiarize the syntax of select SQL statement by incorporating GROUP BY clause.
2. know the importance of GROUP BY clause in selecting records of the table.
3. formulate SQL statements in selecting record/s of the table using GROUP BY clause.



Activating Prior Knowledge

Sample



Presentation of Contents

The SQL GROUP BY Statement

The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country".

The GROUP BY statement is often used with aggregate functions (COUNT, MAX, MIN, SUM, AVG) to group the result-set by one or more columns.

GROUP BY Syntax

SELECT *column_name(s)*
FROM *table_name*

```
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Demo Database

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

SQL GROUP BY Examples

The following SQL statement lists the number of students in each address:

Example

```
SELECT COUNT(StudentID), Address
FROM tblStudents
GROUP BY Address;
```

StudentID	Address
1	Di Masarsarakan City
1	Quezon City

1	Sanchez Mira
1	Null
1	Kabugnayan Village

Another Example

Consider the CUSTOMERS table is having the following records −

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
|  2 | Khilan   |  25 | Delhi     |  1500.00 |
|  3 | kaushik  |  23 | Kota      |  2000.00 |
|  4 | Chaitali |  25 | Mumbai    |  6500.00 |
|  5 | Hardik   |  27 | Bhopal    |  8500.00 |
|  6 | Komal    |  22 | MP        |  4500.00 |
|  7 | Muffy    |  24 | Indore    | 10000.00 |
+----+----------+-----+-----------+----------+
```

If you want to know the total amount of the salary on each customer, then the GROUP BY query would be as follows.

```
SQL> SELECT NAME, SUM(SALARY) FROM CUSTOMERS
  GROUP BY NAME;
```

This would produce the following result −

```
+----------+-------------+
| NAME     | SUM(SALARY) |
+----------+-------------+
| Chaitali |     6500.00 |
| Hardik   |     8500.00 |
| kaushik  |     2000.00 |
| Khilan   |     1500.00 |
| Komal    |     4500.00 |
| Muffy    |    10000.00 |
| Ramesh   |     2000.00 |
+----------+-------------+
```

Now, let us look at a table where the CUSTOMERS table has the following records with duplicate names −

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
|  2 | Ramesh   |  25 | Delhi     |  1500.00 |
```

```
| 3 | kaushik  | 23 | Kota    | 2000.00 |
| 4 | kaushik  | 25 | Mumbai  | 6500.00 |
| 5 | Hardik   | 27 | Bhopal  | 8500.00 |
| 6 | Komal    | 22 | MP      | 4500.00 |
| 7 | Muffy    | 24 | Indore  | 10000.00 |
+----+----------+-----+-----------+----------+
```

Now again, if you want to know the total amount of salary on each customer, then the GROUP BY query would be as follows −

```
SQL> SELECT NAME, SUM(SALARY) FROM CUSTOMERS
  GROUP BY NAME;
```

This would produce the following result −

```
+---------+-------------+
| NAME            | SUM(SALARY) |
+---------+-------------+
| Hardik          |    8500.00 |
| kaushik         |    8500.00 |
| Komal           |    4500.00 |
| Muffy           |   10000.00 |
| Ramesh          |    3500.00 |
+---------+-------------+
```

 **Application**

Sample

 **Feedback**

Sample

Topic 6: Sorting Search Result with ORDER BY

Time Allotment: 1 Hour

 **Learning Objectives**

At the end of the topic, students will be able to:

1. know and familiarize the syntax of select SQL statement by incorporating ORDER BY clause
2. know how to sort the result of searching records in least to greatest, A-Z or its vice versa
3. formulate SQL statements in sorting the result of select SQL statement.

Activating Prior Knowledge

Sample

Presentation of Contents

The SQL ORDER BY Keyword

The ORDER BY keyword is used to sort the result-set in ascending or descending order.

The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

ORDER BY Syntax

```
SELECT column1, column2,                                    ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

Demo Database

Below is demo table "tblStudents" in the dbSIAS sample database:

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE

ORDER BY Example

The following SQL statement selects all students from the "tblStudents" table, sorted by the "Address" column:

Example

```
SELECT * FROM tblStudents
ORDER BY Address;
```

StudentID	StudentNo	StudentName	Address	Gender	Birthday	Course
4	18-12345	Dane Arviel	Null	Null	Null	BSCrim
1	18-14327	Gorgonio Magalpok	Di Masarsarakan City	Male	08/27/2000	BSIT
5	17-23456	Juana Dela Cruz	Kabugnayan Village	Female	02/14/2002	BSE
2	20-02020	Daniel Padilpat	Quezon City	Male	10/13/2000	BSIS
3	19-09278	Eidan Deeandrei	Sanchez Mira	Male	04/27/2001	BMMA

Another Example

Consider the CUSTOMERS table having the following records −

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
|  2 | Khilan   |  25 | Delhi     |  1500.00 |
|  3 | kaushik  |  23 | Kota      |  2000.00 |
|  4 | Chaitali |  25 | Mumbai    |  6500.00 |
|  5 | Hardik   |  27 | Bhopal    |  8500.00 |
|  6 | Komal    |  22 | MP        |  4500.00 |
|  7 | Muffy    |  24 | Indore    | 10000.00 |
+----+----------+-----+-----------+----------+
```

The following code block has an example, which would sort the result in an ascending order by the NAME and the SALARY −

```
SQL> SELECT * FROM CUSTOMERS
  ORDER BY NAME, SALARY;
```

This would produce the following result −

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  4 | Chaitali |  25 | Mumbai    |  6500.00 |
|  5 | Hardik   |  27 | Bhopal    |  8500.00 |
|  3 | kaushik  |  23 | Kota      |  2000.00 |
|  2 | Khilan   |  25 | Delhi     |  1500.00 |
|  6 | Komal    |  22 | MP        |  4500.00 |
|  7 | Muffy    |  24 | Indore    | 10000.00 |
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
+----+----------+-----+-----------+----------+
```

The following code block has an example, which would sort the result in the descending order by NAME.

```
SQL> SELECT * FROM CUSTOMERS
  ORDER BY NAME DESC;
```

This would produce the following result −

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2000.00 |
|  7 | Muffy    |  24 | Indore    | 10000.00 |
|  6 | Komal    |  22 | MP        |  4500.00 |
|  2 | Khilan   |  25 | Delhi     |  1500.00 |
|  3 | kaushik  |  23 | Kota      |  2000.00 |
|  5 | Hardik   |  27 | Bhopal    |  8500.00 |
|  4 | Chaitali |  25 | Mumbai    |  6500.00 |
+----+----------+-----+-----------+----------+
```



Application

Sample



Feedback

Sample

Topic 7: Limiting Search Result with LIMIT

Time Allotment: 1 Hour

Learning Objectives

At the end of the topic, students will be able to:

1. know and familiarize the syntax of select SQL statement by incorporating LIMIT keyword
2. know the importance of LIMIT keyword in selecting record.
3. formulate SQL statements in selecting record/s of the table by limiting search result using LIMIT keyword.

Activating Prior Knowledge

Sample

Presentation of Contents

LIMIT clause is used to limit the data amount returned by the SELECT statement.

Syntax

Following is the basic syntax of SELECT statement with LIMIT clause.

```
SELECT column1, column2, columnN
FROM table_name
LIMIT [no of rows]
```

Following is the syntax of LIMIT clause when it is used along with OFFSET clause.

```
SELECT column1, column2, columnN
FROM table_name
LIMIT [no of rows] OFFSET [row num]
```

SQLite engine will return rows starting from the next row to the given OFFSET as shown below in the last example.

Example

Consider COMPANY table with the following records −

ID	NAME	AGE	ADDRESS	SALARY
1	Paul	32	California	20000.0
2	Allen	25	Texas	15000.0
3	Teddy	23	Norway	20000.0
4	Mark	25	Rich-Mond	65000.0
5	David	27	Texas	85000.0
6	Kim	22	South-Hall	45000.0
7	James	24	Houston	10000.0

Following is an example, which limits the row in the table according to the number of rows you want to fetch from table.

```
sqlite> SELECT * FROM COMPANY LIMIT 6;
```

This will produce the following result.

ID	NAME	AGE	ADDRESS	SALARY
1	Paul	32	California	20000.0
2	Allen	25	Texas	15000.0
3	Teddy	23	Norway	20000.0
4	Mark	25	Rich-Mond	65000.0
5	David	27	Texas	85000.0
6	Kim	22	South-Hall	45000.0

However in certain situations, you may need to pick up a set of records from a particular offset. Here is an example, which picks up 3 records starting from the 3rd position.

```
sqlite> SELECT * FROM COMPANY LIMIT 3 OFFSET 2;
```

This will produce the following result.

ID	NAME	AGE	ADDRESS	SALARY
3	Teddy	23	Norway	20000.0
4	Mark	25	Rich-Mond	65000.0
5	David	27	Texas	85000.0



Application

Sample



Feedback

Sample

References:

https://www.w3schools.com/

https://www.tutorialspoint.com/

https://www.w3resource.com/

https://www.sanfoundry.com/

https://www.sqlitetutorial.net/